BART: Denoising Autoencoder.

BART trains a full seq2seq Transformer to undo damage: corrupt the input with a noise function, then reconstruct the *entire* original text. We derive its cross-entropy objective, work through all five corruption types, and compute every number — the Poisson span distribution, a worked infilling example, and a decoder softmax — in full.

THE EQUATION

$$\mathcal{L}(\theta) \;=\; -\frac{1}{|y|}\sum_{t=1}^{|y|}\log p_\theta\big(y_t \mid y_{<t},\, g(x)\big)$$

Worked example. A 10-token sentence corrupted by text infilling (Poisson(3) span lengths), with the full reconstruction target compared to T5's span-only target.

Read top to bottom: each step feeds the next.

Contents

1 The objective: reconstruct the *whole* sentence

BART is a **denoising autoencoder** built from a standard encoder–decoder Transformer. Training has two halves. First, a stochastic *noise function* $g(\cdot)$ corrupts a clean document x into a damaged version $g(x)$. Second, the model must reconstruct the original x autoregressively. The encoder reads $g(x)$ bidirectionally; the decoder generates x left-to-right, attending to the encoder. The loss is the ordinary sequence-to-sequence negative log-likelihood, averaged over the target tokens:

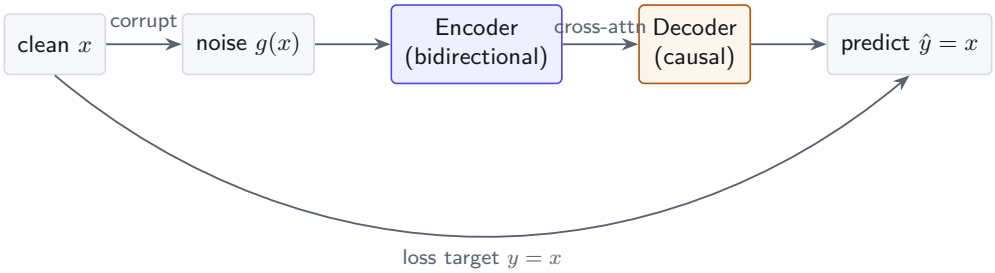
$$\mathcal{L}(\theta) \;=\; -\frac{1}{|y|}\sum_{t=1}^{|y|}\log p_\theta\big(y_t \mid y_{<t},\, g(x)\big), \qquad y \equiv x \tag{1}$$

The defining choice is $y \equiv x$: the target is the *complete* original sequence, not just the corrupted parts. This is what separates BART from masked-span models like T5.

Symbol	Meaning	Shape / type
x	clean original document (also the target y)	token sequence, length $\lvert x\rvert$
$g(x)$	corrupted input produced by the noise function	token sequence, length $\leq \lvert x\rvert$
y_t	target token at decoder step t	token id
$y_{<t}$	target tokens already generated (teacher forcing)	prefix
$p_\theta(\cdot)$	decoder next-token distribution over vocab V	$\in \mathbb{R}^{\lvert V\rvert}$, sums to 1
θ	all encoder + decoder parameters	shared

Think of BART as a printer that has been fed a smudged, shuffled photocopy. Its job is to type out the *entire* clean page from memory, not just fill in the smudges. Because the target is the whole document, every output position contributes a gradient on every step — the learning signal is dense. The encoder is free to be bidirectional (it only *reads* the corruption), while the decoder stays causal (it *writes* the answer). That pairing is exactly an encoder-decoder Transformer.

2 Data flow



The encoder sees only $g(x)$; the original x appears twice on the decoder side — once shifted-right as the teacher-forced input $y_{<t}$, and once as the loss target y.

3 The five corruption types

BART's flexibility comes from mixing several noise functions. Each forces a different kind of reasoning. Let g be drawn (per example) from the schemes below.

1. **Token masking.** Replace random tokens with `[MASK]` (as in BERT). The positions are revealed; the model only fills slots.
2. **Token deletion.** Delete random tokens outright. No placeholder is left, so the model must additionally infer *where* a token was removed — a harder, position-aware task.
3. **Text infilling.** Sample a number of spans; each span of length k (with k drawn from Poisson(λ=3)) is replaced by a *single* `[MASK]`. The model must predict not only the missing content but *how many* tokens are missing. A draw of k=0 inserts a `[MASK]` with no deletion (the model learns to emit *nothing*).
4. **Sentence permutation.** Shuffle the order of sentences; the model restores the original order.
5. **Document rotation.** Choose a uniformly random token and rotate the document so it begins there; the model must identify the true start.

The corruption type controls *what* the model is forced to learn. Masking → content; deletion → content + *location*; infilling → content + *length*; permutation/rotation → global *structure*. Because the target is always the full clean document, one objective (Eq. 1) trains all of these at once.

4 Text infilling and the Poisson(3) span length

Text infilling is BART's signature noise. Span lengths follow

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad \lambda = 3.$$

We compute $P(k)$ for $k = 0, \ldots, 6$ by hand. With $e^{-3} = 0.049787$:

$$P(0) = \tfrac{3^0}{0!}e^{-3} = 1 \cdot 0.049787 = 0.049787, \qquad P(1) = \tfrac{3^1}{1!}e^{-3} = 3 \cdot 0.049787 = 0.149361,$$
$$P(2) = \tfrac{3^2}{2!}e^{-3} = 4.5 \cdot 0.049787 = 0.224042, \qquad P(3) = \tfrac{3^3}{3!}e^{-3} = 4.5 \cdot 0.049787 = 0.224042,$$
$$P(4) = \tfrac{3^4}{4!}e^{-3} = 3.375 \cdot 0.049787 = 0.168031, \qquad P(5) = \tfrac{3^5}{5!}e^{-3} = 2.025 \cdot 0.049787 = 0.100819,$$
$$P(6) = \tfrac{3^6}{6!}e^{-3} = 1.0125 \cdot 0.049787 = 0.050409.$$

k	0	1	2	3	4	5	6
$P(k)$	0.0498	0.1494	**0.2240**	**0.2240**	0.1680	0.1008	0.0504

The distribution peaks (a tie) at $k = 2$ and $k = 3$, has mean $\lambda = 3$ and standard deviation $\sqrt{3} \approx 1.732$. The seven entries above sum to 0.96649; the remaining mass $P(k \geq 7) = 0.03351$ lives in the long right tail (occasional large deletions). The $P(0) = 0.0498$ slot is what teaches the model to handle a `[MASK]` that hides *zero* tokens.

5 Worked example: a 10-token sentence

Take the clean sentence ($|x| = 10$):

$$x = [\,\text{The}_1\ \text{quick}_2\ \text{brown}_3\ \text{fox}_4\ \text{jumps}_5\ \text{over}_6\ \text{the}_7\ \text{lazy}_8\ \text{dog}_9\ \text{today}_{10}\,].$$

Apply text infilling with two sampled spans: span A covers tokens 2, 3 ("quick brown", length k=2) and span B covers token 6 ("over", length k=1). Each span collapses to one `[MASK]`:

$$g(x) = [\,\text{The}\ \texttt{[MASK]}\ \text{fox jumps}\ \texttt{[MASK]}\ \text{the lazy dog today}\,], \qquad |g(x)| = 9.$$

Two `[MASK]` tokens hide three content tokens, so the corrupted input is shorter than the original. The decoder must regenerate the *full* sentence:

$$y = \hat{x} = [\,\text{The quick brown fox jumps over the lazy dog today}\,], \qquad |y| = 10.$$

Contrast with T5. For the same two corrupted spans, T5 replaces each span with a *distinct sentinel* (`<X>`, `<Y>`) and the target contains *only* the spans, terminated by a final sentinel:

$$y_{\text{T5}} = [\,\texttt{<X>}\ \text{quick brown}\ \texttt{<Y>}\ \text{over}\ \texttt{<Z>}\,], \qquad |y_{\text{T5}}| = 6.$$

	BART (this topic)	**T5**
Target content	full original sentence	masked spans only
Target length $\lvert y\rvert$	**10**	6
Tokens supervised per step	all 10 positions	3 content + 3 sentinels
Signal density	high (every word scored)	sparse (only gaps scored)

BART's target is $\approx 1.67\times$ longer here, and it scores positions the model already saw verbatim ("The", "fox", "jumps", ...). That redundancy is deliberate: it yields a denser gradient signal but costs more compute per example.

5.1 Decoder next-token distribution (heatmap)

At the very first decode step ($t = 1$, after the BOS token), suppose the decoder produces logits $z = [3.1,\ 1.2,\ 0.4,\ 2.6,\ -0.5,\ 1.9]$ over a toy candidate set. Subtracting the max (3.1) and applying $p = \mathrm{softmax}(z)$:

$$p = \mathrm{softmax}(z) = [\,0.4647,\ 0.0695,\ 0.0312,\ 0.2819,\ 0.0127,\ 0.1400\,], \qquad \sum p = 1.$$

	The	cat	dog	sat	[M]	on
$p(y_1 \mid \cdot)$	**0.4647**	0.0695	0.0312	0.2819	0.0127	0.1400

The model places 0.4647 on the correct first word "The". Its contribution to Eq. (1) at this step is $-\log 0.4647 = 0.7663$ nats; the full-sequence loss averages such terms over all $|y| = 10$ positions.

6 Properties and consequences

1. **Generality.** Because g can be *any* noise function and the target is always the clean text, BART subsumes BERT-style masking, GPT-style left-to-right generation (with g a prefix mask), and seq2seq denoising under one loss.
2. **Bidirectional encoder, autoregressive decoder.** The encoder is unconstrained by causality (it reads $g(x)$ in full), so it builds rich representations; the decoder is causal and therefore directly usable for generation after fine-tuning.
3. **Length prediction.** Collapsing each span to one `[MASK]` forces the model to decide *how many* tokens to emit per mask — a skill that transfers directly to abstractive summarization, where output length is not given.
4. **Dense supervision.** Reconstructing all $|y|$ tokens (not just gaps) gives a gradient at every position, which the BART paper found especially effective for generation tasks.

Reconstructing the *entire* sequence is more expensive than T5's span-only target. In the worked example BART decodes 10 tokens versus T5's 6, and much of that work re-predicts words the encoder already saw uncorrupted. Per training step the cost scales with $|y| = |x|$, not with the number of masked tokens — so do not expect BART's denoising to be as cheap as a span-only objective on the same corruption rate.

7 Where this sits in the track

	BERT	BART	T5
Architecture	encoder-only	encoder + decoder	encoder + decoder
Pretraining target	masked tokens	*full* original text	masked spans (sentinels)
Corruption	token mask	5 noise functions	span mask (mean ≈ 3)
Decoder	none	causal, autoregressive	causal, autoregressive
Best at	understanding	generation + understanding	unified text-to-text

BERT (Phase 3) gave us the bidirectional encoder; here we wrap it in a decoder to recover a generator. T5 (a later topic) keeps the encoder-decoder shape but switches to span-only targets. BART sits between them: BERT's bidirectional reading + GPT's autoregressive writing.

A Reproducible (NumPy)

Inputs: $\lambda = 3$ for the Poisson span lengths; decoder logits $z = [3.1, 1.2, 0.4, 2.6, -0.5, 1.9]$. The snippet below regenerates the span table and the next-token distribution.

```
import numpy as np
from math import exp, factorial

# --- Poisson(3) span lengths, k = 0..6 ---
lam = 3.0
P = [lam**k * exp(-lam) / factorial(k) for k in range(7)]
print(np.round(P, 6))
# [0.049787 0.149361 0.224042 0.224042 0.168031 0.100819 0.050409]
print(round(sum(P), 5), round(1 - sum(P), 5))   # 0.96649  0.03351  (tail k>=7)

# --- decoder next-token softmax ---
z = np.array([3.1, 1.2, 0.4, 2.6, -0.5, 1.9])
p = np.exp(z - z.max()); p /= p.sum()
print(np.round(p, 4))      # [0.4647 0.0695 0.0312 0.2819 0.0127 0.14  ]
print(round(-np.log(p[0]), 4))   # 0.7663  (loss term for correct token "The")

# --- target-length contrast for the worked example ---
orig_len, bart_target, t5_target = 10, 10, 6
print(bart_target, t5_target, round(bart_target / t5_target, 3))  # 10 6 1.667
```